

Optimizing Dependent Task Offloading with Hop Constraints in Vehicular Edge Computing

Chien-Sheng Chiang¹, SangEun Park², Hao-Tse Hsiao¹, BaekGyu Kim², Chung-Wei Lin¹

¹National Taiwan University (NTU), ²Daegu Gyeongbuk Institute of Science and Technology (DGIST)

Abstract—As the Internet of Vehicles (IoV) continues to evolve, multi-access edge computing (MEC) brings computation devices closer to end users by deploying them at edge computation devices. This proximity enables faster and more reliable computing services. In MEC-enabled IoV networks, task offloading is crucial to reduce the computational load on vehicles and has attracted increasing attention from researchers. However, existing methods typically assume that all computation devices can transmit execution results to each other. In this work, we address a more realistic challenge of offloading subtasks with hop constraints. We divide each task into multiple subtasks with a tree-structured dependency graph. We model the network as a communication graph in which direct transmission between computation devices is only possible when an edge exists, without assuming full connectivity. We then propose a method that maps the dependency graph to the communication graph, and the goal is to minimize the makespan, *i.e.*, the completion time of the last finishing subtask. Experimental results demonstrate that our method achieves a 17.8% average makespan reduction over existing methods across varying system parameters.

Index Terms—Internet of vehicles, multi-access edge computing, task offloading, dependent tasks.

I. INTRODUCTION

The advancement of Internet of Vehicles (IoV) leads to the emergence of various vehicular applications. However, the limited computing and storage resources available in vehicles may lower the service quality of these application or even restrict their executions. One potential solution is to offload computation to the cloud. While cloud computing can provide substantial computational power, its communication latency may be too high for time-critical vehicular applications. Consequently, integrating multi-access edge computing (MEC) into vehicular networks has been recognized as an effective method. MEC deploys edge servers on network devices such as roadside units (RSUs), enabling vehicles to access computing services with low latency and high reliability.

In the early IoV stages, applications were relatively small and monolithic, so computing offloading primarily focused on an entire application, *i.e.*, whether an application should be offloaded and which server it should be offloaded. With the increasing intelligence of vehicles, more computationally complex applications such as driving assistance, autonomous driving, high-definition mapping, and entertainment have emerged. These applications often consist of multiple subtasks that must operate cooperatively to fulfill complex functional requirements. To improve execution parallelism and reduce overall application makespan, subtasks can be offloaded to different computation devices for execution. However, the

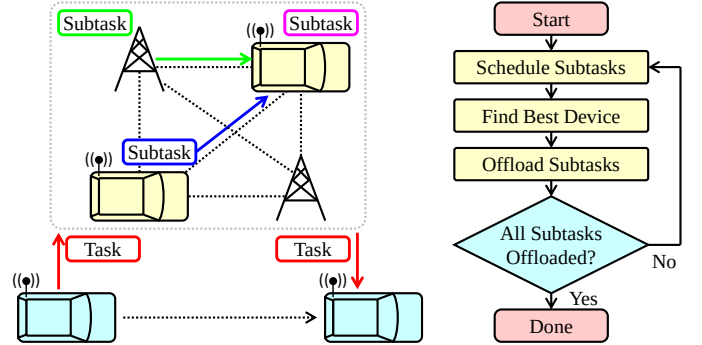


Fig. 1: Subtask offloading and execution work flow. The ego vehicle has to compute red task. It offloads the subtasks (the green, blue, and pink ones) to computation devices around and gets the final result back.

execution priority is constrained by data dependencies, so some existing works [1], [2], [3] combine topological-sort and greedy algorithms to solve the offloading problem.

An example of the subtask offloading and execution work-flow considered in this paper is shown in Fig. 1, where effectively managing subtask dependencies has become a key focus. The offloading problem of subtasks is commonly represented as a directed acyclic graph (DAG), where tasks and their dependencies are depicted as nodes and directed edges, respectively. In this model, each task produces its output only after its completion. The challenge arises when mapping these subtasks to the available computation devices in mobile-edge computing environments, which is an NP-hard problem [4], [5]. The key objective is to minimize the makespan while satisfying dependency constraints under limited and heterogeneous computation devices. The existing methods often adopt genetic algorithms [6] or greedy algorithms [7], [8] as heuristic methods to address this problem.

To combine subtask scheduling and offloading, a common design paradigm is to determine the execution priority of subtasks first, followed by offloading decisions. Representative works following this schedule-then-offload framework can be found in [9], [10], [11], which commonly adopt topological-sort-based methods to determine the offloading priority of subtasks. However, there is one major challenge with such topological-sort-based methods: they implicitly assume that all computation devices can transmit subtask results to each other. In practice, this assumption may not hold due to bandwidth

II. RELATED WORK

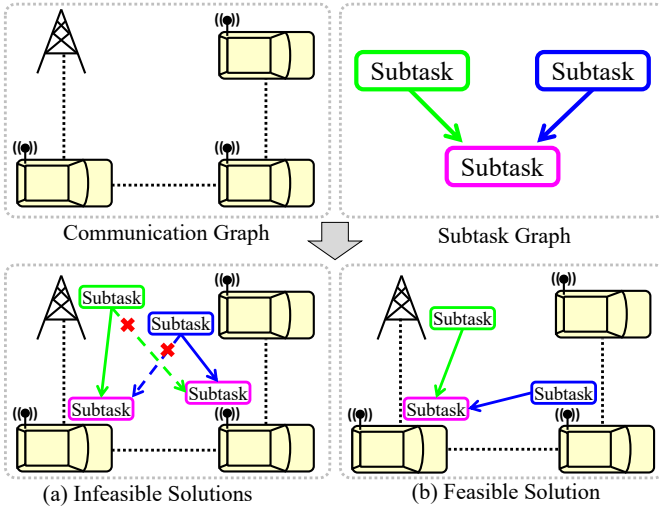


Fig. 2: Given a communication graph and a subtask graph, (a) a topological-sort-based priority assumes that all computation devices can transmit subtask results to each other, leading to infeasible solutions — if the green and blue subtasks have been offloaded, no matter where to offload the pink subtask, it has no connection with either the green or blue subtask; (b) the proposed BFS-based offloading priority guarantees that, when a subtask is offloaded, at most one adjacent subtask has already been offloaded, ensuring feasibility.

limitations and unstable connections, and the existing methods may produce infeasible schedules as shown in Fig. 2.

To address this issue, we propose a breadth-first search-based (BFS-based) method to determine the offloading priority, which explicitly accounts for communication constraints and guarantees feasible schedules in practice, regardless of the network topology.

The main contributions of this paper are as follows:

- 1) We propose a method accounts for multi-hop transmission constraints and incorporates a BFS-based offloading priority to ensure feasible schedules.
- 2) We propose a reassign technique to resolve the inconsistency between BFS-based priority and actual execution priority. We develop a greedy offloading algorithm that offloads subtasks based on the BFS-based priority.
- 3) We evaluate our method against random assignment and existing methods [11] in terms of the makespan. Experimental results demonstrate that our method achieves a 17.8% average makespan reduction over existing methods across varying system parameters.

The remainder of the paper is organized as follows. Section II reviews the related work, while Section III introduces the system model and formulates the problem. The proposed method and the complexity analysis are detailed in Section IV. Simulation results and discussions are presented in Section V. Finally, Section VI concludes the paper and outlines future research directions.

Many studies have investigated task offloading strategies in MEC environments. They mainly focus on minimizing metrics such as delay [12], [13], [14], cost [15], and energy consumption [16]. However, these studies model each task as an independent job and thus do not consider inter-task dependencies. In practical IoV systems, applications are represented as subtasks, where multiple subtasks exhibit complex dependency structures. Therefore, explicitly modeling task dependencies becomes essential.

Task dependency-aware scheduling has been extensively studied in embedded systems even before the emergence of MEC. However, such scheduling studies [1], [2], [3], [17], [18] consider inter-subtask dependencies only at the intra-device level, without accounting for offloading-related issues. Recently, several works have incorporated that issues into MEC systems. Accordingly, some studies [9], [10], [6], [7], [8], [11] model the network node topology as a graph structure. However, they simplify the connectivity model by assuming hop-unconstrained communication, where transmission between two devices is considered feasible as long as a multi-hop path exists. Such an assumption may lead to infeasible offloading decisions in real-world IoV environments, where devices are partially connected and constrained by hop limits.

In addition to connectivity modeling, bandwidth heterogeneity is another key factor in task dependency-aware offloading for IoV systems, as transmission capacities vary across different communication types, such as Vehicle to Vehicle (V2V), Vehicle to MEC server (V2M), and MEC to MEC (M2M). Although [19] considers network conditions in multi-DAG scheduling, it assumes a fixed transmission bandwidth for all communications. Furthermore, [20] does not fully capture diverse offloading scenarios in IoV systems, such as V2V and M2M communications. While [21] investigates DAG scheduling in V2V-based vehicular cloud environments, RSUs are assumed to act solely as controllers and do not participate in task execution.

From the computation perspective, execution-level resource queueing is important aspect of subtask scheduling. Although [22] considers heterogeneous execution times, it does not explicitly model execution-level queueing delays during subtask scheduling. Similarly, [12] considers limited computation capacity at the RSU by allocating CPU processing rates to multiple tasks, it does not explicitly account for computation queueing delays.

By contrast with the above works, our model explicitly considers multiple practical aspects: task dependencies are modeled as a tree structure; the network topology does not to be fully connected; bandwidth heterogeneity is captured by assigning different communication rates to V2V, V2M, and M2M links; and computation queueing is modeled by incorporating the waiting time induced by previously offloaded subtasks on the same device.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. Computation Device Model

We have two kinds of computation devices. V represents the set of vehicles with CPU frequency F_v , where $v \in V$. M represents the set of MECs, which are deployed in RSUs. The CPU frequency of MECs is denoted as F_m , where $m \in M$. We define the set of all computation devices as $C = V \cup M$.

B. Task Model

When vehicles generate subtasks, subtasks should wait for all of the direct precedence subtasks to complete their computation and receive the execution results from all of the direct precedence subtasks. We assume that a vehicle v generates subtasks G , which is represented by a tree structure $G = (N, E)$, where N is the set of subtasks and E is the set of directed edges, which show the dependency between subtasks. The CPU cycle to complete a subtask n is denoted by W_n . The computation time of subtask n executed on device c is

$$\Delta_{n,c} = \frac{W_n}{F_c}, \quad (1)$$

where F_c is the CPU frequency on device c .

An directed edge between two subtasks n and n' is $e_{n,n'}$ and subtask n' can only be executed after receiving the execution results of subtask n . The amount of data to transmit from subtask n to subtask n' is denoted by $S_{n,n'}$, which is the weight of the edge $e_{n,n'}$.

C. Communication Model

There are three different links for the task offloading problem. Vehicle to Vehicle (V2V): For any two vehicles, they can directly communicate with wireless signal if the distance between two vehicles is under coverage $\rho_{v,v'}$. Vehicle to MEC server (V2M): For any vehicles and MECs, they can directly communicate with wireless signal if the distance between the vehicle and the MEC is under coverage $\rho_{v,m}$. MEC to MEC (M2M): For any two MEC servers, they can directly communicate with fiber-optic links if there are links between two servers.

There are two cases for data transmission. For edge $e_{n,n'}$, when subtask n and subtask n' are offloaded to same computation device, the data transmission time is negligible. When two subtasks are not offloaded to the same computation devices, the execution results of subtask n should be sent to the proxy device where the subtask n' is offloaded. When the transmission link is V2V, a single-hop communication rate between vehicle v and v' can be expressed as [23]

$$R_{v,v'} = B_{v,v'} \log_2 \left(1 + \frac{\xi_V P_{v,v'}}{N_0} \right), \quad (2)$$

where $B_{v,v'}$ is the network bandwidth between two vehicles, ξ_V is the transmission power of V2V, $P_{v,v'}$ indicates transmission path loss between vehicle v and vehicle v' , and N_0 is the noise power of the communication link.

When the transmission link is V2M, a single-hop communication rate between vehicle v and MEC server m can be represented as [23]

$$R_{v,m} = B_{v,m} \log_2 \left(1 + \frac{\xi_{V,M} P_{v,m}}{N_0} \right), \quad (3)$$

where $B_{v,m}$ is the network bandwidth in V2M, $\xi_{V,M}$ indicates the transmission power of V2M, and $P_{v,m}$ is transmission path loss between vehicle v and MEC server m .

In this problem, we take

$$P_{v,v'} = 63.3 + 17.7 \log_{10}(D_{v,v'}); \quad (4)$$

$$P_{v,m} = 128.1 + 37.5 \log_{10}(D_{v,m}), \quad (5)$$

where $D_{v,v'}$ is the distance between vehicle v and v' and $D_{v,m}$ is the distance between vehicle v and MEC server m .

When the transmission link is M2M, the communication rate between MEC server m and m' is fixed as $R_{m,m'}$.

We model the topology of communication model as an undirected graph, where each computation device is a node and two devices are connected if they can communicate within H hops. Formally, we first construct an intermediate graph $\mathcal{G}^{\text{raw}} = (C, L^{\text{raw}})$ based on direct communication links, and then derive the equivalent single-hop graph $\mathcal{G} = (C, L)$ by

$$L = \{(c, c') \mid c, c' \in C, d_{\mathcal{G}^{\text{raw}}}(c, c') \leq H\}, \quad (6)$$

where $d_{\mathcal{G}^{\text{raw}}}(c, c')$ is the shortest path distance in $\mathcal{G}^{\text{raw}}$. All subsequent operations are performed on $\mathcal{G}$, and we denote by $\text{Neighbors}(c) = \{c' \in C \mid (c, c') \in L\}$ the neighbors of device c in $\mathcal{G}$.

D. Problem Formulation

An ego vehicle aims to offload some subtasks to computation devices to minimize the makespan. We assume that each subtask is non-preemptively executed on a computation device. The offloading decision can be represented as a two-dimensional ($|N| \times |C|$) binary matrix Y . If subtask n is offloaded to computation device c , then $y_{n,c} = 1$; otherwise, $y_{n,c} = 0$.

Given $y_{n,c} = 1$, the start time is

$$t_{n,c}^{\#} = \max \left(\tau_c, \max_{n' \in \text{Pred}(n), y_{n',c'}=1} \left(t_{n',c'}^* + \frac{S_{n,n'}}{R_{c,c'}} \right) \right), \quad (7)$$

where τ_c indicates the earliest available time of device c , and $\text{Pred}(n) = \{n' \in N \mid (n', n) \in E\}$ denotes the set of predecessor subtasks of n , and the completion time is

$$t_{n,c}^* = t_{n,c}^{\#} + \Delta_{n,c}. \quad (8)$$

The optimization objective is to decide an offloading decision to minimize the makespan:

$$\min \max_{n \in N} t_n^*. \quad (9)$$

Algorithm 1 Execution Priority Assignment

Require: $G, \mathcal{G}$ **Ensure:** Priority vector P

```
1:  $P \leftarrow \mathbf{0}$ ,  $visited \leftarrow \emptyset$ ,  $pending \leftarrow N^{\text{sink}}$ 
2: Construct reversed graph  $G^T$  with edges  $(j, i)$  for each  $(i, j) \in E$ 
3: while  $pending \neq \emptyset$  do
4:   for  $n \in pending$  with all  $\text{Pred}^T(n) \subseteq visited$  do
5:      $P[n] \leftarrow \max_{n' \in \text{Pred}^T(n)} \left\{ P[n'] + \Delta_n + \frac{S_{n',n}}{R_{n',n}} \right\}$ 
6:      $visited \leftarrow visited \cup \{n\}$ 
7:      $pending \leftarrow (pending \setminus \{n\}) \cup \{n'' \in \text{Succ}^T(n) \mid n'' \notin visited\}$ 
8:   end for
9: end while
10: return  $P$ 
```

IV. ALGORITHM DESIGN

In this section, we present a centralized method to solve the objective from the perspective of the ego vehicle, which has a global view of the system information, *i.e.*, G and $\mathcal{G}$. The proposed method is divided into three phases. In the first phase, we determine the execution priority of subtasks based on task dependencies. In the second phase, we determine the offloading priority and prove that the resulting priority assignment does not lead to infeasible schedules. In the third phase, we develop a greedy method to determine the offloading decisions. Specifically, we construct a binary decision matrix Y , where $y_{n,c} = 1$ indicates that subtask n is offloaded to computation device c , and enforce the constraint $\sum_c y_{n,c} = 1$ for all subtasks n . Our objective is to minimize the makespan.

A. Execution Priority Assignment

We now describe the execution priority as shown in Algorithm 1. The priority $P(n)$ must respect the topological priority of G . Following the definition in [11], $P(n)$ is defined as

$$P(n) = \max_{n' \in \text{Succ}(n)} \left(P(n') + \Delta_n + \frac{S_{n,n'}}{R_{n,n'}} \right), \quad (10)$$

where $\text{Succ}(n) = \{n' \mid (n, n') \in E\}$ denotes the set of successor subtasks of n , and Δ_n denotes the average computation time of subtask n across all devices, and $\frac{S_{n,n'}}{R_{n,n'}}$ represents the average transmission time between subtask n and its successor. These are defined as:

$$\Delta_n = \frac{1}{|C|} \sum_{c \in C} \Delta_{n,c}; \quad (11)$$

$$\frac{S_{n,n'}}{R_{n,n'}} = \frac{1}{|L|} \sum_{(c,c') \in L} \frac{S_{n,n'}}{R_{c,c'}}. \quad (12)$$

To compute this efficiently, we construct a reversed subtask graph G^T and traverse it from sink subtasks to root subtasks. This bottom-up approach ensures that when computing the priority of a task n , all priorities of its successors in G (*i.e.*, predecessors in G^T) have already been determined.

The algorithm initializes a pending queue with all sink subtasks in Line 1, and constructs the reversed subtask graph G^T in Line 2. In Lines 3–9, we repeatedly process subtasks from the queue: for each subtask n whose all predecessors in G^T have been visited (Line 4), we compute its priority according to Eqn. (10) in Line 5, add it to the visited set in Line 6, and insert its unvisited successors in G^T (equivalently, predecessors in G) into the queue in Line 7.

B. Offloading Priority Assignment

We first illustrate why a topological-sort-based method may lead to infeasible solutions. As shown in Fig. 2, a topological-sort-based method may assign the blue subtask the highest priority, the green subtask with the medium priority, and the pink subtask with the lowest priority. Following this priority, suppose we offload the blue subtask to the upper right vehicle and the green subtask to the MEC server. In this situation, the pink subtask cannot be offloaded to any computation device, because none of the available devices can simultaneously receive the subtask results from both the upper right vehicle and the MEC server. Thus, the schedules become infeasible under the hop-constrained communication graph.

Definition 1 (Assigned Neighbors). *Given a partial assignment $A \subseteq N$, the assigned neighbors of subtask n are*

$$\mathcal{B}(n) = \{n' \in A \mid (n, n') \in E \text{ or } (n', n) \in E\}.$$

Theorem 1. *If $|\mathcal{B}(n)| \leq 1$ when assigning subtask n , a feasible device assignment for n always exists.*

Proof. If $|\mathcal{B}(n)| = 0$, then n has no communication constraint and can be placed on any device connected with ego vehicle. If $|\mathcal{B}(n)| = 1$, let n' be the unique assigned neighbor on device $\sigma(n')$. Then n requires a device c satisfying $(\sigma(n'), c) \in L$. Such c always exists in $\mathcal{G}$. $\square$

Theorem 2. *When subtasks are processed in BFS order on G , $|\mathcal{B}(n)| \leq 1$ holds for every subtask n .*

Proof. Suppose for contradiction that $|\mathcal{B}(n)| \geq 2$. Let $u, v \in \mathcal{B}(n)$ be two distinct assigned neighbors of n . Since both are assigned before n in BFS, there exists a path $u \rightsquigarrow v$ not through n . Combined with edges (u, n) and (n, v) , this forms a cycle in G , contradicting G being a tree. $\square$

By Theorem 1 and Theorem 2, BFS order guarantees a feasible device assignment at every step.

To determine the offloading priority, we adopt a BFS-based method in Algorithm 2. In Line 1, the subtask n^* with the highest execution priority is identified and pushed into a priority queue PQ in Line 2. In each iteration (Lines 3–15), the subtask n with the highest priority is popped from PQ in Line 4 and record the processed n into $visited$ in Line 5. In Line 6–11, the best device is selected greedily by minimizing the makespan via Algorithm 3 (which will be described in Section IV-C). Finally, all unvisited neighbors of n in G are pushed into PQ in Lines 12–13.

Algorithm 2 BFS-Based Subtask Offloading**Require:** $G, \mathcal{G}$, execution priority P, c^{ego} **Ensure:** $Y, \max_{n \in N} t_n^*$

```

1:  $n^* \leftarrow \arg \max_{n \in N} P(n), p^* \leftarrow P(n^*)$ 
2:  $\text{push}(PQ, (p^*, n^*, \emptyset))$ 
3: while  $PQ \neq \emptyset$  do
4:    $(p, n, src) \leftarrow \text{pop}(PQ)$ 
5:    $\text{push}(\text{visited}, (p, n))$ 
6:   if  $src = \emptyset$  then
7:      $\mathcal{C} \leftarrow \text{Neighbors}(c^{ego})$ 
8:   else
9:      $\mathcal{C} \leftarrow \text{Neighbors}(Y[src])$ 
10:  end if
11:   $Y[n] \leftarrow \arg \min_{c \in \mathcal{C}} \text{Reassignment}(\text{visited}, n, c, Y)$ 
12:  for  $n' \in \text{Neighbor}(n)$  with  $n' \notin \text{visited}$  do
13:     $\text{push}(PQ, (-P(n'), n', n))$ 
14:  end for
15: end while
16: return  $Y, \max_{n \in N} t_n^*$ 

```

Algorithm 3 Reassignment**Require:** $\text{visited}, n_{\text{cur}}, c, Y$ **Ensure:** $\max_{n \in N} t_n^*$

```

1: Initialize  $t_n^\# \leftarrow 0, t_n^* \leftarrow 0, \tau_c \leftarrow 0$  for all  $n \in N, c \in \mathcal{C}$ 
2:  $V \leftarrow \{n \mid (p, n) \in \text{visited}\}$ 
3:  $\hat{Y}[n] \leftarrow \begin{cases} c & \text{if } n = n_{\text{cur}} \\ Y[n] & \text{otherwise} \end{cases}$ 
4: while  $\text{visited} \neq \emptyset$  do
5:    $(p, n) \leftarrow \text{pop}(\text{visited})$ 
6:   for  $n' \in \text{Pred}(n) \cap V$  do
7:      $t_n^\# \leftarrow \max\left(t_n^\#, t_{n'}^* + \frac{S_{n',n}}{R_{\hat{Y}[n'], \hat{Y}[n]}}\right)$ 
8:   end for
9:    $t_n^\# \leftarrow \max\left(t_n^\#, \tau_{\hat{Y}[n]}\right)$ 
10:   $t_n^* \leftarrow t_n^\# + \Delta_{n, \hat{Y}[n]}$ 
11:   $\tau_{\hat{Y}[n]} \leftarrow t_n^*$ 
12: end while
13: return  $\max_{n \in N} t_n^*$ 

```

C. Subtask Offloading

As shown in Algorithm 2, subtasks are processed sequentially by offloading priority. In Lines 6–9, the candidate devices are determined by the source subtask src : if n is the first subtask, the candidates $\mathcal{C}$ are the neighbors of the ego vehicle c^{ego} in $\mathcal{G}$ (Line 7); otherwise, they are restricted to the neighbors of the device assigned to src (Line 9). Then, in Line 11, for each candidate device c , we evaluate the current makespan for processed subtask by assuming that subtask n is offloaded to c , and select the device that minimizes this value. Since the execution priority may differ from the offloading priority, the current makespan must be recomputed based on the execution priority of all previously offloaded subtasks.

The current makespan is computed by a reassignment procedure in Algorithm 3. In Line 2, we maintain a set V

TABLE I: Default Values of Simulation Parameters

Graph	Parameter	Value
Subtask Graph G	F_v	2 GHz
	F_m	10 GHz
	W	$[1.6, 10] \times 10^6$
	$ N $	500
	$B_{v,v'}$	25 MHz
Communication Graph $\mathcal{G}$	$B_{v,m}$	30 MHz
	ξ_V	23 dBm
	ξ_M	46 dBm
	N_0	-174 dBm/Hz
	$R_{m,m'}$	1 Gbps
	$S_{n,n'}$	$[8, 50] \times 10^6$ bytes
	$ V $	50
	$ M $	10
	H	4
	$\rho_{v,v'}$	150m
	$\rho_{v,m}$	200m
	ℓ	1500m (side length)

recording all subtasks in visited before any pop occurs, so that $\text{Pred}(n) \cap V$ in Line 6 correctly identifies the already-assigned predecessors of n . In Line 3, we define a temporary assignment $\hat{Y}$ that overrides the assignment of n_{cur} with candidate device c , while keeping all other assignments unchanged. For each subtask n popped in priority order (Line 5), its start time $t_n^\#$ is computed in Lines 6–9 by taking the maximum over all data dependencies from assigned predecessors (Line 7) and the earliest available time of its assigned device (Line 9). The completion time t_n^* is then obtained in Line 10, and the earliest available time of device is updated in Line 11. The makespan is returned in Line 13.

D. Computation Complexity Analysis

We analyze the time complexity of our method in two phases. In the first phase, we determine the execution priority by accessing each subtask node and edge once, resulting in a time complexity of $O(|N| + |E|)$, where $|N|$ is the number of subtasks, and $|E|$ is the number of edges in the subtask graph G . In the second phase, we offload all subtasks, giving an initial complexity of $O(|N|)$. For each subtask, we evaluate all available computation devices, which increases the complexity to $O(|N||C|)$, where $|C|$ is the number of computation devices in the communication graph $\mathcal{G}$. Additionally, to find the best computation device, we traverse all previously offloaded subtasks and their parent subtasks, which requires $O(|N| + |E|)$ time. Therefore, the overall time complexity of the method is $O(|N||C|(|N| + |E|))$.

V. SIMULATION RESULTS AND DISCUSSIONS**A. Settings**

We implement the simulation using Python 3.11. The default values of the parameters of the subtask graph and communication graph are summarized in Table I, similar to [11], [23]. We consider a square area with side length ℓ , in which vehicles are deployed uniformly at random, and MEC servers are placed to maximize coverage. The ego vehicle is selected uniformly at random in each trial. The subtask graph G is generated as a

TABLE II: Makespan (s) under Varying Simulation Parameters (The default values of the parameters are shown in Table I.)

Parameter	Value	Makespan (s) ↓		
		PBTSA	Random	Ours
Side Length (ℓ m)	500	0.32	1.72	0.31
	1000	0.47	1.86	0.38
	1500	0.61	1.58	0.52
	2000	0.69	1.52	0.58
Number of Subtasks ($ N $)	100	0.24	1.04	0.11
	250	0.40	1.50	0.29
	400	0.61	1.58	0.52
	600	0.84	2.13	0.75
Number of Vehicles ($ V $)	10	0.81	1.48	0.71
	50	0.61	1.58	0.52
	75	0.52	1.89	0.45
	100	0.52	1.84	0.43
Numebe of MECs ($ M $)	1	0.76	1.32	0.64
	5	0.63	1.60	0.52
	10	0.61	1.58	0.52
	20	0.52	1.91	0.42
Maximum Hops (H)	1	0.67	0.99	0.52
	2	0.65	1.21	0.52
	4	0.61	1.58	0.52
	6	0.61	1.78	0.53

random tree with $|N|$ subtasks. All results are averaged over 20 independent trials with different random subtask graph G and communication graph $\mathcal{G}$.

We evaluate all methods using the makespan, defined as $\max_{n \in N} t_n^*$, *i.e.*, the completion time of the last finishing subtask. Specifically, we compare our proposed method against the following baselines:

- **PBTSA** [11]: A topological-sort-based method combined with a maximum clique algorithm to extract a fully connected subgraph from $\mathcal{G}$, ensuring that a feasible solution always exists.
- **Random**: Our BFS-based priority strategy with random device selection. The priority ensures feasibility, while the offloading device is chosen uniformly at random from the candidate set instead of greedily.

B. Results and Discussion

In this section, we evaluate the performance of the proposed method using the makespan as the evaluation metric. All results are averaged over 20 independent trials to reduce the impact of randomness. Table II presents the makespan under varying system parameters, where the base configuration parameters follows Table I.

As the side length ℓ increases from 500m to 2000m, vehicles are spread over a larger area, resulting in a sparser communication graph $\mathcal{G}$ with fewer connected device pairs and greater physical distances. Consequently, the candidate set for each subtask shrinks and transmission latency increases, causing the makespan to grow for all methods. Our method consistently achieves the lowest makespan across all settings, with improvements of 3.1%, 19.1%, 14.8%, and 15.9% over PBTSA at ℓ is 500m, 1000m, 1500m, and 2000m, respectively.

As the number of subtasks increases from 100 to 600, all methods exhibit increasing makespan due to greater computation and communication overhead. Our method maintains

a consistent advantage, achieving 54.2%, 27.5%, 16.1%, and 10.7% reduction over PBTSA at 100, 250, 400, and 600 subtasks, respectively. While the gap narrows as task size grows, our method maintains an absolute advantage across all settings, consistently achieving the lowest makespan.

Increasing the number of vehicles from 10 to 100 consistently reduces the makespan for all methods, as more vehicles provide additional computation devices and denser communication graph. Our method reduces the makespan by 12.3%, 14.8%, 13.5%, and 17.3% over PBTSA at 10, 50, 75, and 100 vehicles, respectively. Notably, the improvement of our method over PBTSA grows with the number of vehicles, suggesting that our method can better exploits the additional computation devices.

As the number of MEC servers increases from 1 to 20, the makespan decreases for all methods due to higher computation capacity and better load distribution. Our method achieves reductions of 15.8%, 17.5%, 14.8%, and 19.2% over PBTSA at 1, 5, 10, and 20 MEC servers, respectively. The improvement is most pronounced at a larger number of MEC servers, indicating that our method more effectively utilizes the increased offloading opportunities.

As H increases from 1 to 6, all methods benefit from richer multi-hop paths, but the gains quickly saturate beyond $H = 4$. Notably, our method is relatively insensitive to H , maintaining a makespan around 0.52s. In contrast, PBTSA shows a stronger dependency on H , with its makespan decreasing more noticeably as H grows. Nevertheless, our method consistently outperforms PBTSA across all hop settings, with improvements of 22.4%, 20.0%, 14.8%, and 13.1% at H are 1, 2, 4, and 6, respectively.

We further analyze the behavior of the Random baseline across different parameter settings. As the number of subtasks increases, the makespan grows for all methods due to greater computation and communication overhead, and the Random baseline is most affected since suboptimal device selections accumulate over more subtasks. As the side length ℓ increases, the communication graph becomes sparser, shrinking the candidate set and leaving fewer opportunities for poor random choices, which partially explains why the gap between Random and the other methods narrows at larger ℓ . In contrast, increasing the number of vehicles, MEC servers, or maximum hops all expand the candidate set, providing more offloading options. While our greedy method consistently exploits this larger search space to minimize the makespan, the Random baseline suffers from the fact that a larger candidate set increases the probability of selecting a suboptimal device, resulting in higher makespan. This is evidenced by the consistently increasing makespan of the Random baseline as $|V|$, $|M|$, and H grow, while other methods consistently benefit from these changes.

VI. CONCLUSION

In this paper, we present a method for offloading subtasks to computation devices with the objective of minimizing the

makespan when the communication graph is partially connected. The main challenge arises from the fact that existing topological-sort-based methods assume a fully connected communication graph, which may not hold in practical vehicular networks. Our proposed method addresses this by leveraging a breadth-first search (BFS) for offloading priority assignment, a topological-sort-based algorithm for execution priority assignment, and finally a greedy algorithm to offload subtasks to minimize the makespan. Experimental results show that our method consistently outperforms a topological-sort-based baseline (PBTSA) and a random offloading strategy, achieving an average makespan reduction of 17.8% over PBTSA across varying configurations. Furthermore, we show that while a larger candidate set benefits our greedy strategy, it degrades random selection, highlighting the importance of informed device assignment in partially connected communication graph. Future work includes extending the method to account for vehicle mobility and general DAG task dependencies.

REFERENCES

- [1] W. Zheng, C. Wang, Z. Chen, and D. Zhang, "A priority-based level heuristic approach for scheduling DAG applications with uncertainties," in *IEEE International Conference on Computer Supported Cooperative Work in Design*, 2022, pp. 1022–1027.
- [2] G. Cordasco, R. De Chiara, and A. L. Rosenberg, "An area-oriented heuristic for scheduling dags on volatile computing platforms," *IEEE Transactions on Parallel and Distributed Systems*, vol. 26, no. 8, pp. 2164–2177, 2015.
- [3] G. Malewicz, I. Foster, A. Rosenberg, and M. Wilde, "A tool for prioritizing dagman jobs and its evaluation," in *IEEE International Conference on High Performance Distributed Computing*, 2006, pp. 156–168.
- [4] M. Kaur, "Fastpga based scheduling of dependent tasks in grid computing to provide qos to grid users," in *International Conference on Internet of Things and Applications*, 2016, pp. 418–423.
- [5] P. Belotti, C. Kirches, S. Leyffer, J. Linderoth, J. Luedtke, and A. Mahajan, "Mixed-integer nonlinear optimization," *Acta Numerica*, vol. 22, p. 1–131, 2013.
- [6] M.-A. Hassan Abdel-Jabbar, I. Kacem, and S. Martin, "Unrelated parallel machines with precedence constraints: application to cloud computing," in *IEEE International Conference on Cloud Networking*, 2014, pp. 438–442.
- [7] M. Jia, J. Cao, and L. Yang, "Heuristic offloading of concurrent tasks for computation-intensive applications in mobile cloud computing," in *IEEE International Conference on Computer Communications Workshops*, 2014, pp. 352–357.
- [8] Y.-H. Kao, B. Krishnamachari, M.-R. Ra, and F. Bai, "Hermes: Latency optimal task assignment for resource-constrained mobile computing," vol. 16, no. 11, pp. 3056–3069, 2017.
- [9] S. Sundar and B. Liang, "Offloading dependent tasks with communication delay and deadline constraint," in *IEEE International Conference on Computer Communications*, 2018, pp. 37–45.
- [10] X. Lv, H. Du, and Q. Ye, "TBTOA: A DAG-based task offloading scheme for mobile edge computing," in *IEEE International Conference on Communications*, 2022, pp. 4607–4612.
- [11] Z. Wang, G. Sun, H. Su, H. Yu, B. Lei, and M. Guizani, "Low-latency scheduling approach for dependent tasks in MEC-enabled 5G vehicular networks," vol. 11, no. 4, pp. 6278–6289, 2024.
- [12] Q. Luo, C. Li, T. H. Luan, and W. Shi, "Minimizing the delay and cost of computation offloading for vehicular edge computing," *IEEE Transactions on Services Computing*, vol. 15, no. 5, pp. 2897–2909, 2022.
- [13] C. Chen, L. Chen, L. Liu, S. He, X. Yuan, D. Lan, and Z. Chen, "Delay-optimized v2v-based computation offloading in urban vehicular edge computing and networks," *IEEE Access*, vol. 8, pp. 18 863–18 873, 2020.
- [14] Z. Ning, K. Zhang, X. Wang, L. Guo, X. Hu, J. Huang, B. Hu, and R. Y. K. Kwok, "Intelligent edge computing in internet of vehicles: A joint computation offloading and caching solution," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 4, pp. 2212–2225, 2021.
- [15] J. Huang, J. Wan, B. Lv, Q. Ye, and Y. Chen, "Joint computation offloading and resource allocation for edge-cloud collaboration in internet of vehicles via deep reinforcement learning," vol. 17, no. 2, pp. 2500–2511, 2023.
- [16] K. Zhang, Y. Mao, S. Leng, Q. Zhao, L. Li, X. Peng, L. Pan, S. Maharjan, and Y. Zhang, "Energy-efficient offloading for mobile edge computing in 5G heterogeneous networks," vol. 4, pp. 5896–5907, 2016.
- [17] Z. Hu, D. Li, Y. Zhang, D. Guo, and Z. Li, "Branch scheduling: dag-aware scheduling for speeding up data-parallel jobs," in *International Symposium on Quality of Service*, ser. IWQoS '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: <https://doi.org/10.1145/3326285.3329071>
- [18] Y. Xu, L. Liu, and Z. Ding, "Dag-aware joint task scheduling and cache management in spark clusters," in *IEEE International Parallel and Distributed Processing Symposium*, 2020, pp. 378–387.
- [19] Y. Fan, L. Zhai, and H. Wang, "Cost-efficient dependent task offloading for multiusers," *IEEE Access*, vol. 7, pp. 115 843–115 856, 2019.
- [20] Q. Shen, B.-J. Hu, and E. Xia, "Dependency-aware task offloading and service caching in vehicular edge computing," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 12, pp. 13 182–13 197, 2022.
- [21] Z. Liu, M. Liwang, S. Hosseinalipour, H. Dai, Z. Gao, and L. Huang, "Rfid: Towards low latency and reliable dag task scheduling over dynamic vehicular clouds," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 9, pp. 12 139–12 153, 2023.
- [22] L. Liu, H. Tan, S. H.-C. Jiang, Z. Han, X.-Y. Li, and H. Huang, "Dependent task placement and scheduling with function configuration in edge computing," in *International Symposium on Quality of Service*, ser. IWQoS '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: <https://doi.org/10.1145/3326285.3329055>
- [23] B. Li, F. Chen, Z. Peng, P. Hou, and H. Ding, "Mobility-aware dynamic offloading strategy for c-v2x under multi-access edge computing," vol. 49, p. 101446, 2021.